Politecnico di Bari

PROJECT WORK

EMBEDDED CONTROL

WATER TANK CONTROL WITH STM32 MICROCONTROLLER

Professor:

Prof. Luca De Cicco

Students:

Abbaticchio Michele, Matr. 586320

Camposeo Antonio, Matr. 586729

Ceglie Fabio, Matr. 586799

Fonseca Christian, Matr. 587152

Sommario

1. Project Goal

The goal of the project is to implement a control system using an STM32 microcontroller to regulate the water level in a tank. The project involves construction of the physical system, modelling of the mathematical system and controller, and related software implementations in the microcontroller.

The road map of the project is explained in the following picture:

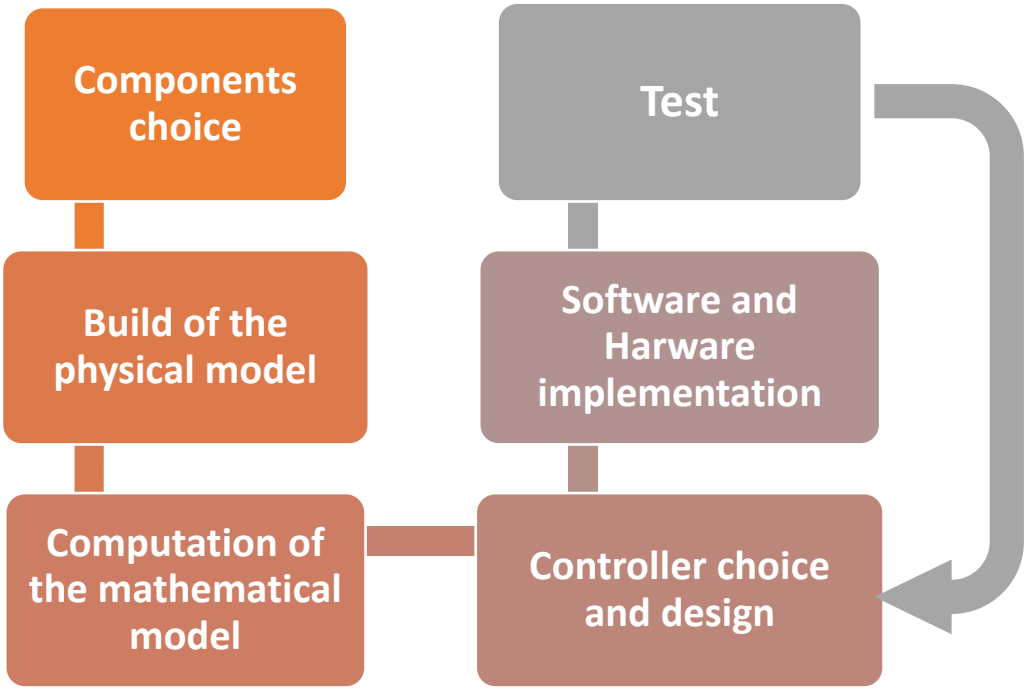


Figure 1 Workflow

2. Design Choices

The design choices for this project are the microcontroller, the numbers and the models of sensors and actuators. The microcontroller chosen is the STM32 NUCLEO F411RE, as it can provide us with all the GPIOs and timers we need for controller implementation and input and output management. For the sensor, a laser sensor was searched: this choice was driven by the necessity to have a fast and accurate sensor, with an interface and a library compatible with an STM32 NUCLEO board: the VL53L0X provides all those features. Two of these sensors will be placed on the top of the two tanks to get the water level in each tank at every clock time. For the pump, the choice was driven by the tank's volumes and the desired settling time of the plant: a pump driven by a 12V motor can provide an output flow capable to fill a 5 L tank in small times. Also, is compatible with an L298N Dual H-Bridge, which is the driver chosen for his compatibility with our microcontroller and pump.

3.Sensor

The VL53L0X is a new generation Time-of-Flight (ToF) laser-ranging, providing accurate distance measurement whatever the target reflectance's unlike conventional technologies. It can measure absolute distances up to 2m, setting a new benchmark in ranging performance levels, opening the door to various new applications.

The VL53L0X integrates a leading-edge SPAD array (Single Photon Avalanche Diodes) and embeds ST's second generation FlightSenseTM patented technology.

The VL53L0X's 940 nm VCSEL emitter (Vertical Cavity Surface-Emitting Laser), is totally invisible to the human eye, coupled with internal physical infrared filters, it enables longer ranging distances, higher immunity to ambient light, and better robustness to cover glass optical crosstalk.[1]

Technical specification:

Feature	Detail
Package	Optical LGA12
Size	4.40 x 2.40 x 1.00 mm
Operating voltage	2.6 to 3.5 V
Operating temperature:	-20 to 70°C
Infrared emitter	940 nm
I^2C	Up to 400 kHz (FAST mode) serial bus Address: 0x52

Figure 2 Sensor Technical Specification

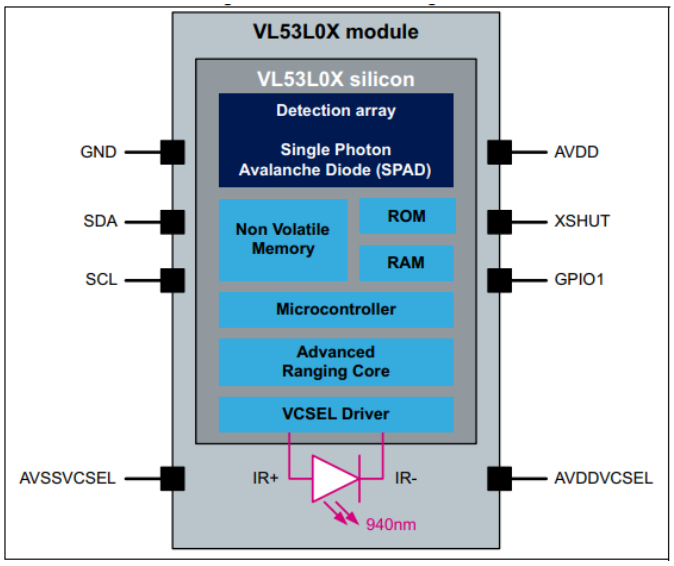


Figure 3 System Block Diagram

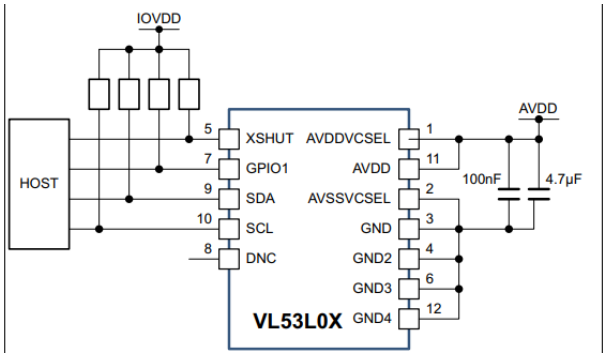


Figure 4 VL53L0X Schematic

3.1 Functional Description

The host customer application is controlling the VL53L0X device using an API (Application Programming Interface). The API is exposing to the customer application a set of high-level functions that allows control of the VL53L0X Firmware (FW) like initialization/calibration, ranging Start/Stop, choice of accuracy, choice of ranging mode. The API is a turnkey solution, it consists of a set of C functions which enables fast development of end user applications, without the complication of direct multiple register access. The API is structured in a way that it can be compiled on any kind of platform through a well isolated platform layer. The API package allows the user to take full benefit of VL53L0X capabilities. A detailed description of the API is available in the VL53L0X API User Manual (separate document, DocID029105). VL53L0X FW fully manages the hardware (HW) register accesses.[1]

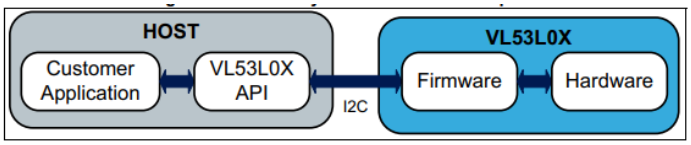


Figure 5 API Diagram

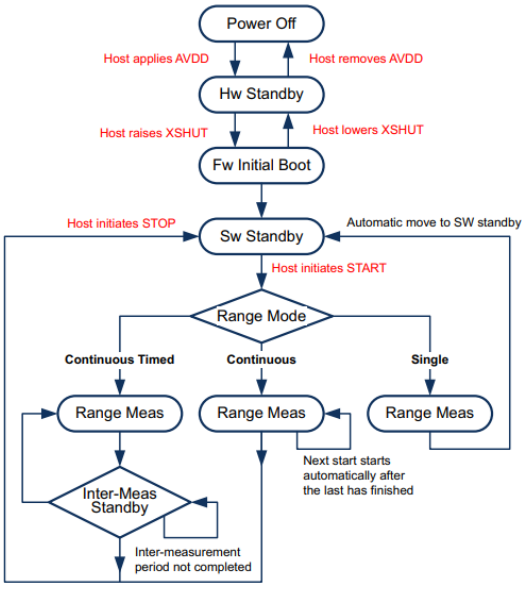


Figure 6 Firmware state machine description

3.2 Power Sequence

There are two options available for device power up/boot.

Option 1: XSHUT pin connected and controlled from host. This option helps to optimize power consumption as the VL53L0X can be completely powered off when not used, and then woken up through host GPIO (using XSHUT pin). HW Standby mode is defined as the period when AVDD is present and XSHUT is low.

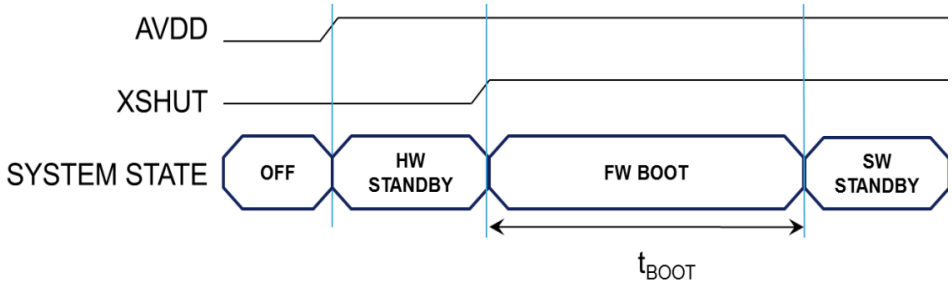


Figure 7 Option 1

Option 2: XSHUT pin not controlled by host, and tied to AVDD through pull-up resistor. In case XSHUT pin is not controlled, the power up sequence is presented in Figure 11. In this case, the device is going automatically in SW STANDBY after FW BOOT, without entering HW STANDBY.

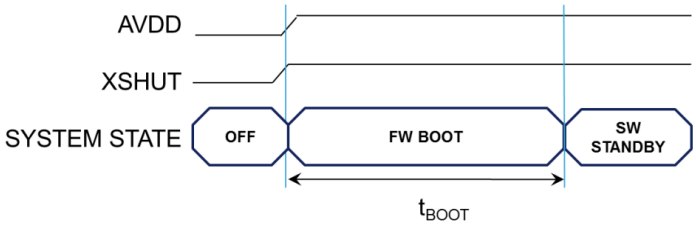


Figure 8 Option 2

In our case we use option 1, which allows us to control 2 same sensors connected to two different XSHUT pin.

3.2.1 Ranging Sequence

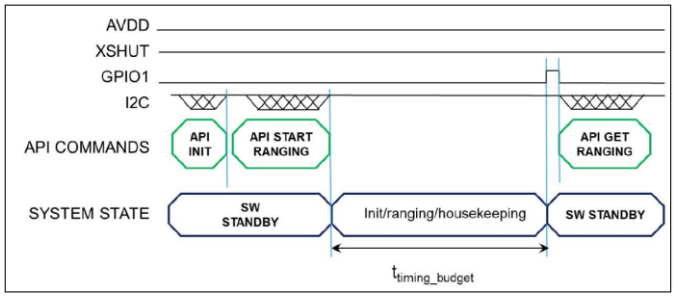


Figure 9 Sequence Acquisition

timing budget is a parameter set by the user, using a dedicated API function. Default value is 33ms.

3.2.2 Ranging Operating Modes

There are 3 ranging modes available in the API:

Single ranging: Ranging is performed only once after the API function is called. System returns to SW standby automatically.

Continuous ranging: Ranging is performed in a continuous way after the API function is called. As soon as the measurement is finished, another one is started without delay. User must stop the ranging to return to SW standby. The last measurement is completed before stopping.

Timed ranging: Ranging is performed in a continuous way after the API function is called. When a measurement is finished, another one is started after a user defined delay. This delay (inter-measurement period) can be defined through the API.

User must stop the ranging to return to SW standby. If the stop request comes during a range measurement, the measurement is completed before stopping. If it happens during an inter-measurement period, the range measurement stops immediately.[1]

3.2.3 Ranging Profiles

Four different ranging profiles are available via API example code. Customers can create their own ranging profile dependent on their use case performance requirements. For more details, please refer to the VL53L0X API User Manual.[2]

1. Default mode

2. High speed

3. High accuracy

4. Long range

Each range profile consists of 3 consecutive phases:

• Initialization and load calibration data

• Ranging

• Digital housekeeping

For more details, refer to the VL53L0X – Datasheet [1].

Getting data and control interface

User can get the final data using a polling or an interrupt mechanism.

Polling mode: user must check the status of the ongoing measurement by polling an Api function.

Interrupt mode: An interrupt pin (ex GPIO1) sends an interrupt to the host when a new measurement is available.

The description of these two modes is written in details in [2].

Device physical control interface is I2C. The I2C interface uses two signals: serial data line (SDA) and serial clock line (SCL). Each device connected to the bus is using a unique address and a simple master / slave relationships exists. Both SDA and SCL lines are connected to a positive supply voltage using pull-up resistors located on the host. Lines are only actively driven low. A high condition occurs when lines are floating, and the pull-up resistors pull lines up. When no data is transmitted both lines are high.

Clock signal (SCL) generation is performed by the master device. The master device initiates data transfer. The I2C bus on the VL53L0X has a maximum speed of 400 kbits/s and uses a device address of 0x52.

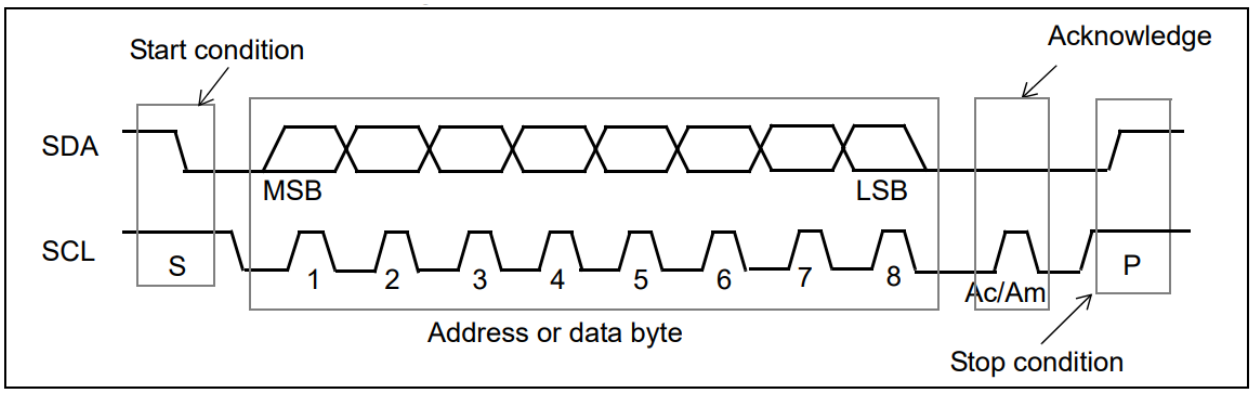


Figure 10 I2C Data Transfer

Information is packed in 8-bit packets (bytes) always followed by an acknowledge bit, Ac for VL53L0X acknowledge and Am for master acknowledge (host bus master). The internal data is produced by sampling SDA at a rising edge of SCL. The external data must be stable during the high period of SCL. The exceptions to this are start (S) or stop (P) conditions when SDA falls or rises respectively, while SCL is high.

A message contains a series of bytes preceded by a start condition and followed by either a stop or repeated start (another start condition but without a preceding stop condition) followed by another message. The first byte contains the device address (0x52) and specifies the data direction. If the least significant bit is low (that is, 0x52) the message is a master write to the slave. If the LSB is set (that is, 0x53) then the message is a master read from the slave.

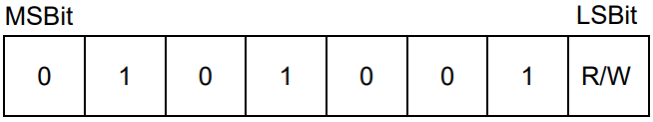


Figure 11 I2C device address 0x52

All serial interface communications with the Time-of-Flight sensor must begin with a start condition. The VL53L0X module acknowledges the receipt of a valid address by driving the SDA wire low. The state of the read/write bit (lsb of the address byte) is stored and the next byte of data, sampled from SDA, can be interpreted. During a write sequence the second byte received provide a 8-bit index which points to one of the internal 8-bit registers.

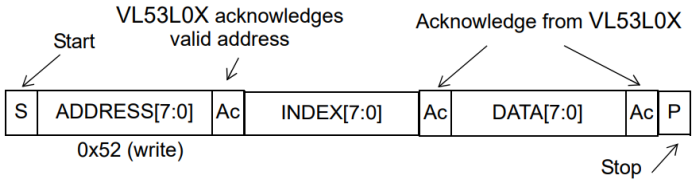


Figure 12 VL53L0X data format (write)

At the end of each byte, in both read and write message sequences, an acknowledge is issued by the receiving device (that is, the VL53L0X for a write and the host for a read). A message can only be terminated by the bus master, either by issuing a stop condition or by a negative acknowledge (that is, not pulling the SDA line low) after reading a complete byte during a read operation.

The interface also supports auto-increment indexing. After the first data byte has been transferred, the index is automatically incremented by 1. The master can therefore send data bytes continuously to the slave until the slave fails to provide an acknowledge or the master terminates the write communication with a stop condition. If the auto-increment feature is used the master does not have to send address indexes to accompany the data bytes.

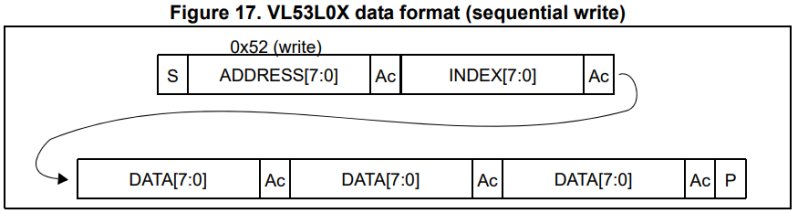


Figure 13 VL53L0X data format (sequential write)

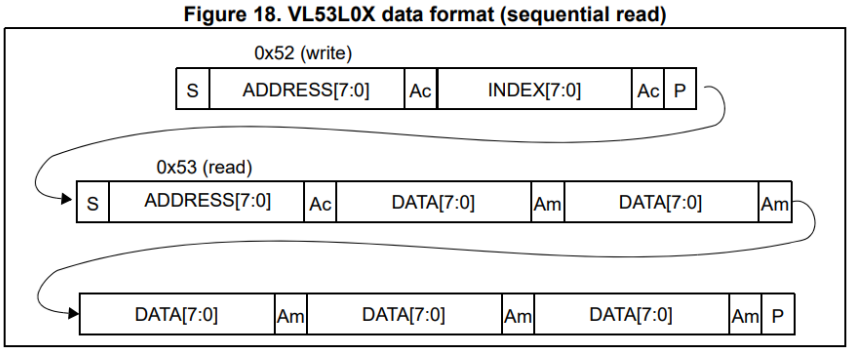


Figure 14 VL53L0X data format (sequential read)

3.3 Multiple Sensors Initialization

Since the VL53L0X can have the I2C device address changed by doing an I2C write once it is booted, a unique reset pin would be needed for each VL53L0X used in a design. Each device is then taken out of reset one at a time, and then the I2C Device Address is changed to a new unique address. This can be done by using multiple GPIO pins from the microprocessor on the board.

To set multiple sensors in code:

• Set VL53L0x_SINGLE_DEVICE_DRIVER macro to 0 so that API implementation will be automatically adapted to a multi-device context.

• Define VL53L0xDev_t type as a structure pointer holding any data required for multidevice management. A mandatory field is an instance of VL53L0xDevData containing ST API private data.

• Then:

- define, “N” the number of VL53L0X (Struct MyVL53L0xDev_t BoardDevs[N])

- Put all devices under reset.

- Enable VL53L0X one after the other and set their I2C address throughVL53L0X_SetDeviceAddress (&BoardDevs[i], FinalI2cAddr)

Code:

In file sensors.c/.h there are functions for initialize and config sensors. First of all, we need to set a struct for sensors setting.

```
struct sensors{
    VL53L0X_RangingMeasurementData_t RangingData[sensor_qty];
    VL53L0X_Dev_t  vl53l0x_c[sensor_qty];
    VL53L0X_DEV Dev_array[sensor_qty];
    };
```

Figure 15 Struct defined in sensor.h

```
/* definition of sensor struct*/
struct sensors array_sensor;
array_sensor.Dev_array[0] = &array_sensor.vl53l0x_c[0];
array_sensor.Dev_array[1] = &array_sensor.vl53l0x_c[1];
```

Figure 16 Struct variable in main.c

```
/*Sensors Initialization*/
array_sensor.Dev_array[0]->I2cHandle = &hi2c1;
array_sensor.Dev_array[1]->I2cHandle = &hi2c1;
array_sensor.Dev_array[0]->I2cDevAddr =  array_sensor.Dev_array[1]->I2cDevAddr = original_addr;
printf("Sensor Init:\n\r");
Sensor_Init(array_sensor,HIGH_ACCURACY); // function from sensors.h
```

Figure 17 Sensor Init in main.c

```
void Sensor_Init(struct sensors s,sensor_modes mode){

    HAL_GPIO_WritePin(GPIOC, arr_gpio[0], GPIO_PIN_RESET);  // Disable XSHUT
    HAL_GPIO_WritePin(GPIOC, arr_gpio[1], GPIO_PIN_RESET);  // Disable XSHUT
    HAL_Delay(10);

    for (int i = 0; i<2;i++){

        HAL_GPIO_WritePin(GPIOC, arr_gpio[i], GPIO_PIN_SET);        // Enable XSHUT
        HAL_Delay(20);

        VL53L0X_WaitDeviceBooted(s.Dev_array[i]);
        VL53L0X_DataInit((s.Dev_array[i]));
        if (i == 0){
            printf("Addr change 1: %i \n\r\n\r", VL53L0X_SetDeviceAddress(s.Dev_array[i], sensor1_addr));
            s.Dev_array[i]->I2cDevAddr = sensor1_addr;
        }else{
            printf("Addr change 2: %i \n\r\n\r", VL53L0X_SetDeviceAddress(s.Dev_array[i], sensor2_addr));
            s.Dev_array[i]->I2cDevAddr = sensor2_addr;
        }
        Sensor_Config(s.Dev_array[i],mode);
    }
}
```

Figure 18 Function Sensor Init in sensors.c

```
void Sensor_Config(VL53L0X_DEV Dev,sensor_modes mode){

    VL53L0X_WaitDeviceBooted( Dev );
    VL53L0X_StaticInit( Dev );
    VL53L0X_PerformRefCalibration(Dev,  &VhvSettings, &PhaseCal);
    VL53L0X_PerformRefSpadManagement(Dev, &refSpadCount, &isApertureSpads);
    VL53L0X_SetDeviceMode(Dev, VL53L0X_DEVICEMODE_SINGLE_RANGING);

    switch(mode){
                    case 0:
                        printf("LONG RANGE");
                        VL53L0X_SetLimitCheckValue(Dev,VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE,(FixPoint1616_t)(0.1*65536));
                        VL53L0X_SetLimitCheckValue(Dev,VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE,(FixPoint1616_t)(60*65536));
                        VL53L0X_SetMeasurementTimingBudgetMicroSeconds(Dev,33000);
                        VL53L0X_SetVcselPulsePeriod(Dev, VL53L0X_VCSEL_PERIOD_PRE_RANGE, 18);
                        VL53L0X_SetVcselPulsePeriod(Dev, VL53L0X_VCSEL_PERIOD_FINAL_RANGE, 14);
                        break;
                    case 1:
                        printf("HIGH SPEED");
                        VL53L0X_SetLimitCheckValue(Dev,VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE,(FixPoint1616_t)(0.25*65536));
                        VL53L0X_SetLimitCheckValue(Dev,VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE,(FixPoint1616_t)(32*65536));
                        VL53L0X_SetMeasurementTimingBudgetMicroSeconds(Dev, 20000);
                        break;
                    case 2:
                        printf("HIGH ACCURACY");
                        VL53L0X_SetLimitCheckValue(Dev,VL53L0X_CHECKENABLE_SIGNAL_RATE_FINAL_RANGE,
                        (FixPoint1616_t)(0.25*65536));
                        VL53L0X_SetLimitCheckValue(Dev,VL53L0X_CHECKENABLE_SIGMA_FINAL_RANGE,(FixPoint1616_t)(18*65536));
                        VL53L0X_SetMeasurementTimingBudgetMicroSeconds(Dev,200000);
                        break;

    }
    HAL_Delay(100);
}
```

Figure 19 Function Sensor_Config in sensors.c

After having initialized and configured both sensors, we are read to acquire data and calculate the liquid level into the tank. As recommended by the datasheet each sensor have a unique address, in our case VL53L0X_1 on 0x54, and VL53L0X_2 on 0x56.

In Figure 17, which shows the configuration function, we may notice 3 different range profiles for sensor, that are LONG RANGE, HIGH SPEED and HIGH ACCURACY. For this project sensors are setup on HIGH ACCURAY, since the settling time is quite large, around 20 second so the sampling

time is around 2-5 seconds. By using this sensor setup measurements take more time but, in our case, we can afford it, obtaining more precision.

These 3 settings are provided by API Range Profiles in [2].

To get data from API functions which communicate with sensors, has been added Get_Data_Sensors (Figure 20).

```
void Get_Data_Sensors(struct sensors s, uint32_t* data1,uint32_t* data2){

        /*Check that these are not pointing to NULL*/
        assert(data1);
        assert(data2);

        if((VL53L0X_PerformSingleRangingMeasurement(s.Dev_array[0], &s.RangingData[0]) == 0) ){

            if( (s.RangingData[0].RangeStatus == 0)  ){
                *data1 = s.RangingData[0].RangeMilliMeter;

            }
            else{
                printf("STATUS1: %i\n\r",  s.RangingData[0].RangeStatus);
            }
        }

        if((VL53L0X_PerformSingleRangingMeasurement(s.Dev_array[1], &s.RangingData[1]) == 0) ){

                if( (s.RangingData[1].RangeStatus == 0)  ){
                    *data2 = s.RangingData[1].RangeMilliMeter;
                }
                else{
                    printf("STATUS2: %i\n\r", s.RangingData[1].RangeStatus);
                }
            }


}
```

Figure 20 Function Get_Data_Sensors in sensors.c

This function assign at data1 and data2 address, the values of measurement allocated in the struct. This measurement will be used to calculate level of liquid, by subtracting these values at the height of tank expressed in millimetres.

```
void MapLevel(uint32_t *data1,uint32_t *data2,Tank_Param_t* tank1,Tank_Param_t* tank2,float* lev1,float* lev2)
{
    *lev1 = tank1->sensor_height - (float)(*data1)/10;
    *lev2 = tank2->sensor_height - (float)(*data2)/10;
}
```

Figure 21 Map Level Function

This function takes as input sensors structures, the data sensed by the sensor, and the parameters of the tanks, and provides the height of the liquid in both tanks calculated as the difference from the height of the sensor and the data sensed.

4. Sensor Testing

The basic principle of a ToF sensor is to measure the distance to the target based on the time for emitted photons to be reflected. In most applications, photons travel through the air. But, to measure the level of a liquid, photons travel through air and water. As a result, the light path is impacted by the different refractive indices.

In other words, there is part of the signal that is reflected from the surface, and another part returned from the container's base (traveling through the liquid), and another part bounces from the liquid itself. Thus, the final ranging distance is impacted with respect to accuracy.

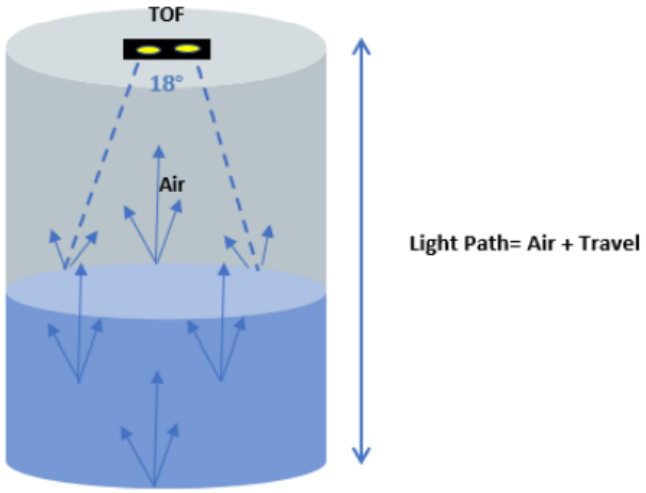


Figure 22 Different light paths when ToF signal aimed at liquid

From [3], there's some recommendation to follow about containers proprieties and reflections.

The position of the VL53L0X sensor and the container size impact the accuracy in ranging because the sensor's field of view (FoV) is cone shaped (25° for VL53L0X). The ToF should be mounted so that the signals are reflected only from the water surface, and not from the edge of the container. If the container is narrow, with a diameter less than the ToF's FoV (as illustrated in Figure 2. FoV bigger than container diameter) then the signals are reflected from the edge, which induces noise. This impacts the ranging accuracy. Therefore, ideally, the container diameter should be greater than the ToF's FoV (as illustrated in Figure 3. FoV smaller than container diameter).

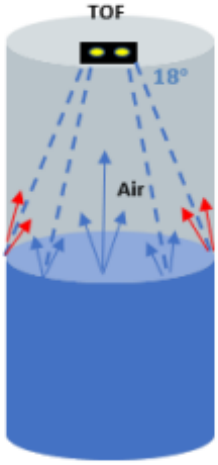


Figure 23 Small Container

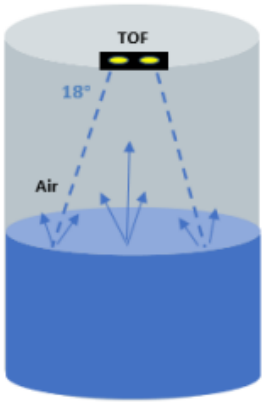


Figure 24 Large Container

In addition, there should be a reasonable gap (minimum 30 mm) between the sensor and the maximum water level. This avoids any obstruction of the ToF's light path, in the case water drops bounce while the water is pouring, or if the container is shaken.

It is recommended to use containers with a base made of low reflective rather than fully transparent or high. This avoids receiving mixed signals from base and liquid surface when the liquid level is low. Water level measurement with different coloured base and no compensation illustrates the ranging behaviour with different background base.

Figure 25 Law reflective Base and Transparent Reflective Base

In conclusion, with these recommendations given by [3] accuracy of measurement increase, but there's another problem related to the type of liquid used. In our case we use water, which is not enough dense to give us an accurate measure. Therefore, it was necessary to add tempera paints and salt to increase the denseness of water.

4.1 Result

Tests have been made to prove that sensors accuracy increase in liquid with more density. These are the result.

Liquid: Water

	Volume (liter)	Height of Sensor (mm)	Liquid Level Mesured by hand (mm)	Liquid Level Mesured by sensor(mm)	Error (mm)
TEST 1	1	275	74 ± 1	63.5 ± 5	-10.5
TEST 2	1.5	275	103 ± 1	111 ± 5	8
TEST 3	2	275	134 ± 1	161 ± 5	27
TEST 4	2.5	275	163 ± 1	179 ± 5	16

Liquid: Water + (30g x Liter) Salt + (8g) Tempera paints

	Volume (liter)	Height of Sensor (mm)	Liquid Level Mesured by hand (mm)	Liquid Level Mesured by sensor(mm)	Error (mm)
TEST 1	1	275	74 ± 1	80 ± 3	6
TEST 2	1.5	275	103 ± 1	101 ± 3	-3
TEST 3	2.2	275	141 ± 1	137 ± 3	5
TEST 4	2.8	275	173 ± 1	178 ± 3	5

The error on measurements of liquid level by sensors is based on 10 measure takes one each second, in both proves it was not static value, but it swings around an interval, which differed from the mean value of the results by 5 in the first case, and by 3 in the second case. The setup of sensor was high accuracy.

5. Pump

The ad20p-1230c pump is capable of a maximum flow of 240L/H and a max head of 3 meters, which is a suitable value for our application. The DC motor that drives the pump has a rated voltage of 12V DC, an operating voltage in the range of 5-12V and a rated current of 0.40A.

Figura 26 AD20P-1230C Pump

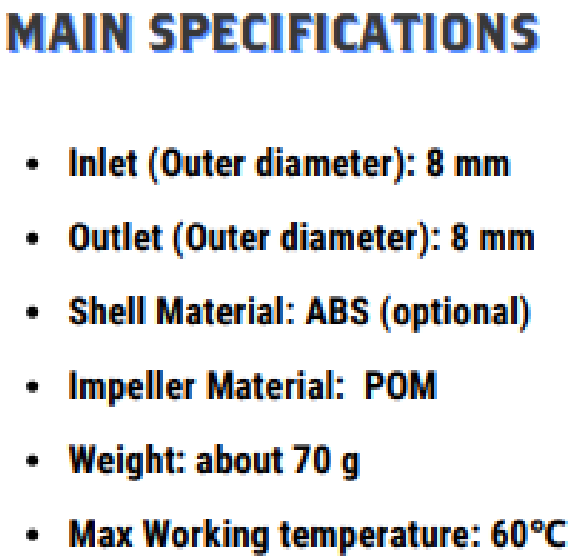


Figura 17 Pump Specs

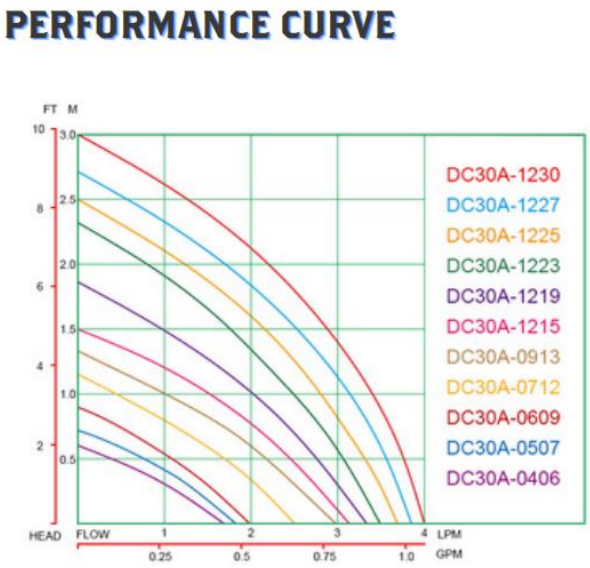


Figura 28 Pump Performance Curve

6. Driver

To drive the pump in voltage, a driver was used to connect the microcontroller with the pump. The HW-095 driver has been identified, which is composed of the input and output pins connected to an L298N bridge, which deals with the translation of the PWM signal into voltage to be applied to the pump motor.

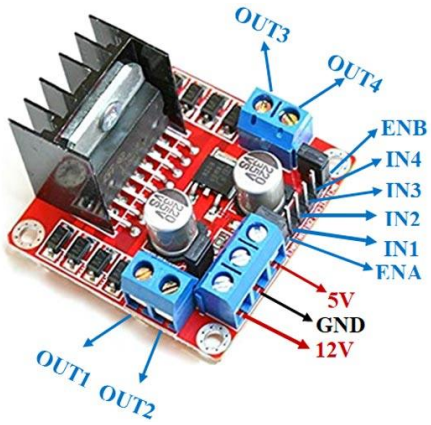


Figure 29 HW-095 Driver

OUT1 and OUT2: Outputs related to the first motor;

OUT3 and OUT4: Outputs relative to the second motor;

ENA, IN1 and IN2: enable and inputs related to the first motor;

ENB, IN3 and IN4: enable and inputs related to the first motor;

12V and GND: input power supply;

5V: output power supply;

The driver can perform the actuation for two motors simultaneously: the output can implement an inductive load in a common or differential mode, depending on the state of the inputs.

The In inputs set the bridge state when The EN input is high; a low state of the EN input inhibits the bridge. All the inputs are TTL (Transistor-Transistor Logic) compatible.

5.1 Settings Pump

To drive the pump, is needed to implement a PWM signal that will go in the EN1 pin of our driver. To do this, we need to set a timer properly: the timer chosen is the TIMER2 of our board, which can provide a PWM signal using an internal clock source. The timer is linked to the APB2 Bus. Furthermore, two GPIO Output needs to be implemented to activate the driver or to select the spinning direction (clockwise or counterclockwise).

IN1	IN2	EVENT
LOW	LOW	NOT SPINNING
LOW	HIGH	CLOCKWISE
HIGH	LOW	COUNTER-CLOCKWISE
HIGH	HIGH	NOT SPINNING

We chose to drive the pump with a PWM frequency equal to 10 kHz, to not introduce resonance in the plant. For a DC motor, we can choose a frequency that satisfies this relation [4]:

$$f_s \geq \frac{5}{2\pi\tau}$$

where τ is the electrical time constant (L/R). In fact, our PWM frequency must be high enough so that the motor, which is essentially a low pass filter, averages out our input voltage, which is a square wave. To implement a PWM signal with this frequency, we needed to know the frequency of the APB bus related to the timer that we wanted to use: the timer 2 was linked to the APB2 bus, which runs at 84 MHz. So, to implement a timer with a 10 kHz frequency, the ARR (Auto Reload Register) and the PSC (Prescaler) values were initialized respectively to 4199 and 1, to satisfy this relation:

$$f_{PWM} = \frac{f_{APB2}}{(ARR + 1)(PSC + 1)}$$

Those values of ARR and PSC have been chosen keeping the PSC value as lowest as possible, to reduce the error and to increase the Duty Cycle precision.

5.2 Actuation Code

```
void actuation_pump(Pump_t* pump, float DC){

    pump->PowerCurrent = (DC);
    if(pump->PowerCurrent > 0){
        TIM2->CCR1 = ((pump->PowerCurrent)/100)*(4199 + 1);
    }
    else{
        TIM2->CCR1 = 0;
    }
}
```

Figure 30 Actuation_pump fuction

This function takes as input the value of the DC computed by one of the implemented controllers. This value is converted in the actual value to pass to the .Pulse register, according to the max value of .AutoReloadRegister, then a saturator limits the value in the admissible range, and then this value is written in the .Pulse register of the timer 2.

5.3 Test Pump

An empirical test was taken in order to check the output flow of the pump at different PWM settings: the objective of this test was to draw the characteristics of the pump related to the DC imposed. A script for this test has been implemented in our NUCLEO F411RE: this script provided to the driver an increasing value of DC, from 0 %to 100%; each value of DC drove the pump for 5 seconds, then the output was weighed to compute the flow in mL/s.

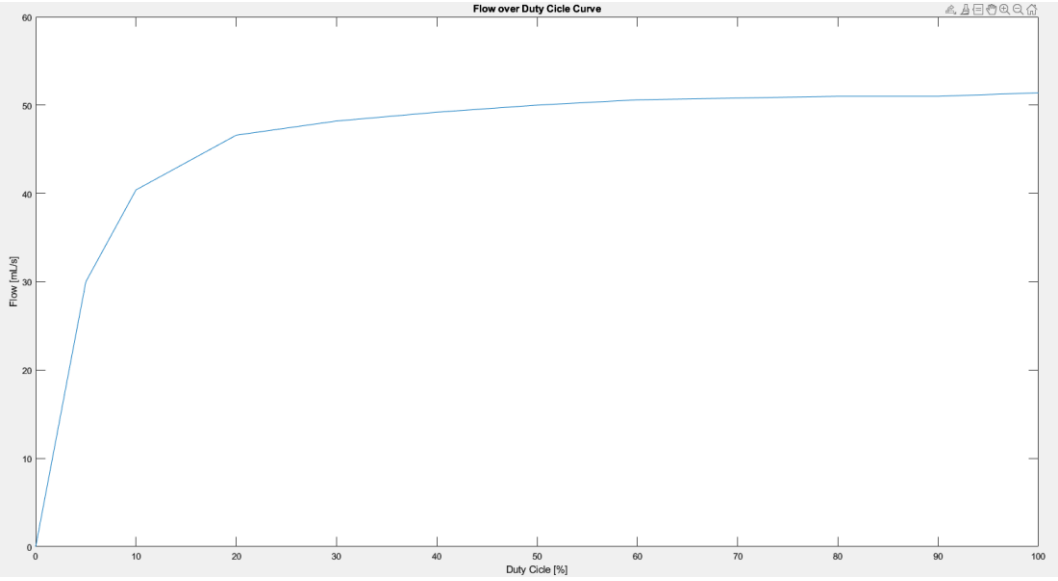


Figure 31 Flow over Duty Cycle curve

7. Real Plant

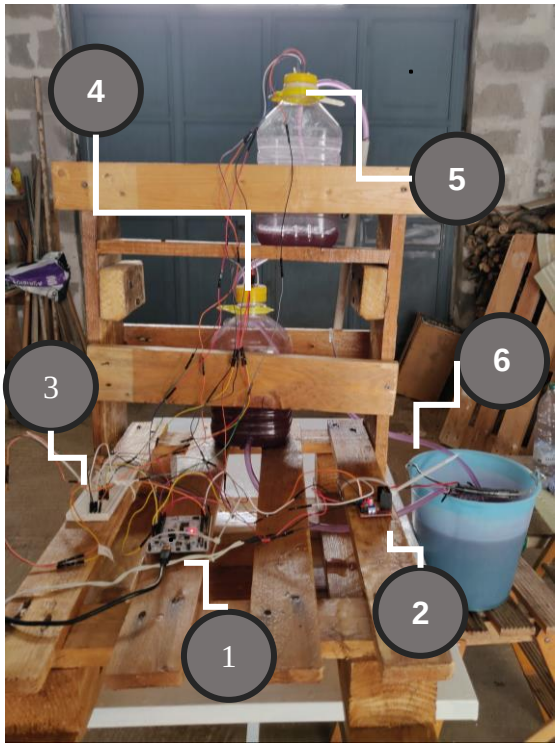


Figure 32 Real Plant photo

1. **Nucleo Stm32 F411re Microcontroller**
2. **Hw-095 Driver**
3. **Breadboards And Wirings**
4. **Tank 2 (And Relative Sensor)**
5. **Tank 1 (And Relative Sensor)**
6. **Basin**

Mathematical Modeling Of The Plant

A system of two tanks is considered:

Parameter	Description	Measure
$\mathbf{A_1}$	Section of the Tank 1	225 cm^2
$\mathbf{h_{t_1}}$	Total height of the Tank 1	28.5 cm
$\mathbf{V_1}$	Total volume of the Tank 1	3000 cm^3
$\mathbf{h_{s_1}}$	Hole height on Tank 1	1.5 cm
$\mathbf{s_1}$	Hole section on Tank 1	0.5025 cm^2
$\mathbf{A_2}$	Section of the Tank 2	225 cm^2
$\mathbf{h_{t_2}}$	Total height of the Tank 2	28.5 cm
$\mathbf{V_2}$	Total volume of the Tank 2	3000 cm^3
$\mathbf{h_{s_2}}$	Hole height on Tank 2	1.5 cm
$\mathbf{s_2}$	Hole section on Tank 2	0.1963 cm^2
$\mathbf{g}$	Gravity acceleration	981 cm/s^2

Based on the model are identified two state variables:

- x_1: fluid level in the tank T_1;
- x_2: fluid level in the tank T_2;

The flow rate of water discharged from a single tank is expressed by the formula:

$$Q_i = \frac{V_i}{dt} = s_i \cdot \frac{x_i}{dt} = s_i \cdot v_i$$

Considering the Torricelli's law, which states that the velocity of water flowing out of a hole in a tank is equivalent to the velocity of fluid falling freely from a height equal to the level of the fluid surface above the hole:

$$Q_i = s_i \cdot \sqrt{2 \cdot g \cdot x_i} = K_i \cdot \sqrt{x_i}$$

Where K_i is the discharge coefficient, which depends on the dimensions of the hole:

$$K_i = s_i \cdot \sqrt{2g}$$

The volume equilibrium of the first tank is expressed as the following formula:

$$V_1(t) = -K_1\sqrt{x_1(t)} + u(t)$$

By isolating the variation of fluid, the formula becomes:

$$A_1 \cdot x_1(t) = -K_1\sqrt{x_1(t)} + u(t)$$

$$x_1(t) = -\frac{K_1}{A_1}\sqrt{x_1(t)} + \frac{1}{A_1}u(t)$$

Similarly, the fluid level variation for the second tank is described by the formula:

$$x_2(t) = \frac{K_1}{A_2}\sqrt{x_1(t)} - \frac{K_2}{A_2}\sqrt{x_2(t)}$$

In conclusion, the mathematical model of the system is:

$$\begin{cases} \dot{x_1}(t) = -\frac{K_1}{A_1}\sqrt{x_1(t)} + \frac{1}{A_1}u(t) \\ \dot{x_2}(t) = \frac{K_1}{A_2}\sqrt{x_1(t)} - \frac{K_2}{A_2}\sqrt{x_2(t)} \end{cases}$$

$$y(t) = x_2(t)$$

The state equations of the system are nonlinear due to the square root.

Equilibrium Points and Linearization

Taking in account the state equations of the model, the existence of the equilibrium states corresponding to a constant input $\overline{u}$ is calculated:

$$\begin{cases} 0 = -\frac{K_1}{A_1}\sqrt{x_1} + \frac{1}{A_1}u \\ 0 = \frac{K_1}{A_2}\sqrt{x_2} - \frac{K_2}{A_2}\sqrt{x_2} \end{cases} \rightarrow \begin{cases} 0 = -K_1\sqrt{x_1} + u \\ 0 = K_1\sqrt{x_2} - K_2\sqrt{x_2} \end{cases} \rightarrow \begin{cases} x_1 = \frac{u^2}{K_1^2} \\ x_2 = \frac{u^2}{K_2^2} \end{cases}$$

It is assumed that the constant input is equal to the maximum flow that enters the second tank. Therefore, is assumed that $u = 39.1\ cm^3/s$ such that the equilibrium states are as follows:

$$\begin{cases} x_1 = 3.0859 \\ x_2 = 20.2215 \end{cases}$$

Due to the nonlinearity of the system, linearization is performed around the equilibrium point. Jacobian matrices are evaluated by computing the partial derivative of the state equations with respect to the state variables and input variable.

$$F = \left(\frac{\delta f}{\delta x}\right)_{\overline{x},\overline{u}} = \begin{bmatrix} -0.02816 & 0 \\ 0.02816 & -0.004297 \end{bmatrix} \quad G = \left(\frac{\delta f}{\delta u}\right)_{\overline{x},\overline{u}} = \begin{bmatrix} 0.004444 \\ 0 \end{bmatrix}$$

$$H = \left(\frac{\delta y}{\delta x}\right)_{\overline{x},\overline{u}} = [0 \quad 1] \quad L = \left(\frac{\delta y}{\delta u}\right)_{\overline{x},\overline{u}} = 0$$

Thus, a linearized model of the two-tank system is obtained from the Jacobian matrices:

$$\begin{cases} \dot{x_1}(t) = -0.02816 \cdot x_1(t) + 0.004444 \cdot u \\ \dot{x_2}(t) = 0.02816 \cdot x_1(t) - 0.004297 \cdot x_2(t) \end{cases}$$

$$y(t) = x_2(t)$$

Due to Lyapunov criterion, the linearized model results asymptotically stable because the eigenvalues of the A matrix are placed in the left side of the plane:

$$\lambda_1 = -0.0043; \lambda_2 = -0.0282$$

The transfer function of the open loop system is:

$$G(s) = \frac{0.0001251}{s^2 + 0.03245s + 0.000121}$$

8. Controller's Logic

The aim of this study is to design a control system capable of achieving a reference signal by actuating a pump based on the data provided by sensors.

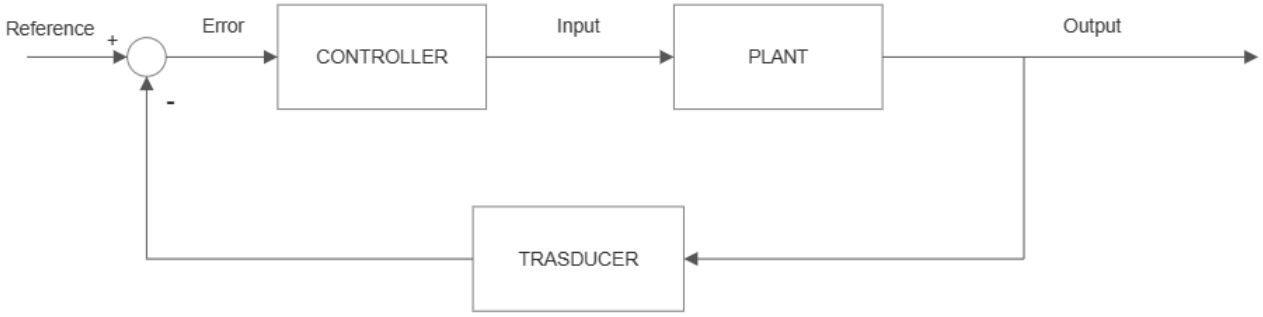


Figure 33 Diagram Block

The system is formed by an analog and a digital part, that are connected via D-A and A-D converters. The analog part comprises of the actuation and controlled process. The digital part is the algorithm of the controller computed on the STM32. The A-D converter is represented by an ideal sampler. The D-A converter is represented by a sampler followed by a zero-order-hold (ZOH). [5]

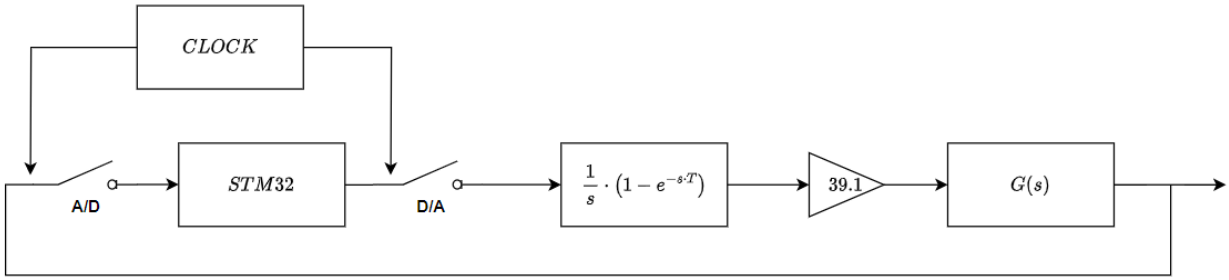


Figure 34 A/D and D/A converters

The zero-order-hold transfer function is approximated with Padé approximation. Additionally, the actuation sampling period is assumed to be 1 second.:

$$H_0(s) \approx \frac{T}{\frac{T}{2}s + 1} = \frac{1}{0.5s + 1}$$

Taking into account the transfer function of the open-loop system G(s), considering the negative phase shift introduced by the ZOH, and taking into consideration the maximum actuation flow rate value of 39.1 cm^3/s, the aggregate continuous transfer function F(s) is:

$$F(s) = 39.1 \cdot H_0(s)G(s) = 39.1 \cdot \frac{1}{0.5s + 1} \cdot \frac{0.0001251}{s^2 + 0.03245s + 0.000121}$$

$$F(s) = \frac{0.004893}{0.75s^3 + 1.024s^2 + 0.03254s + 0.000121}$$

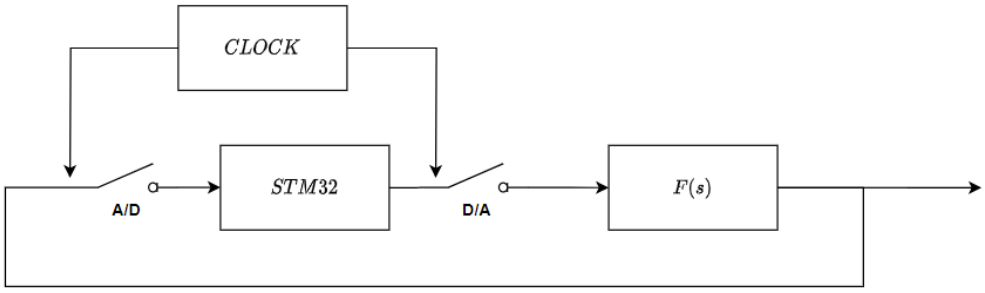
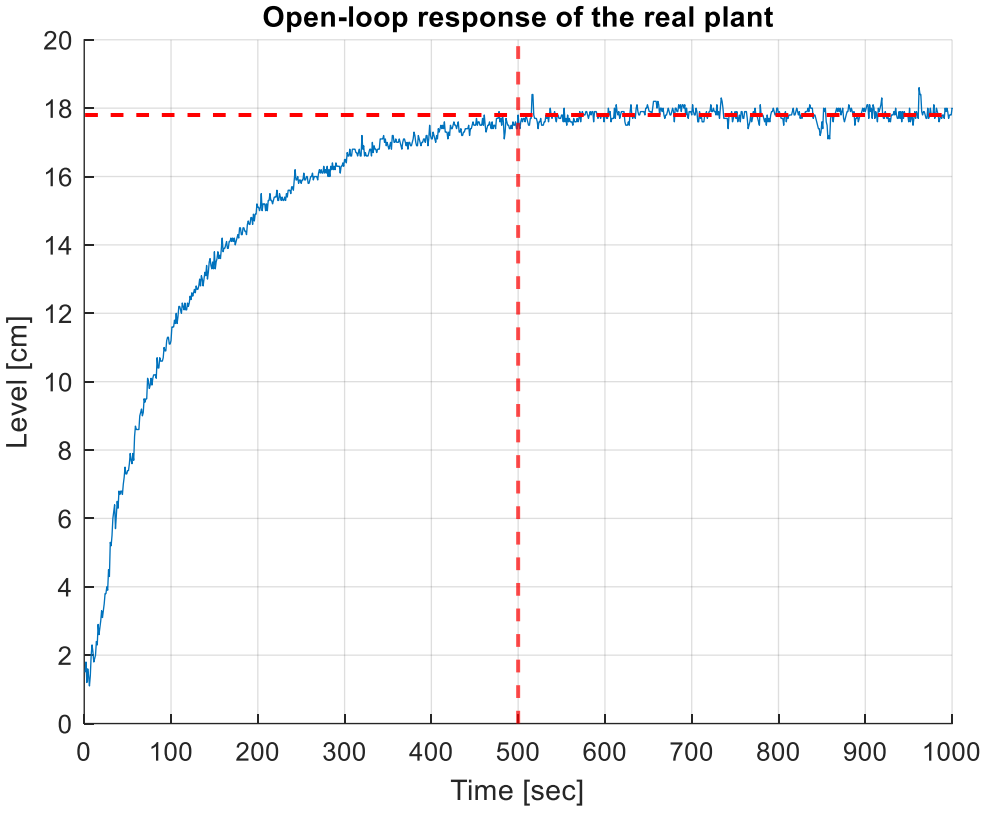


Figure 35 A/D and D/A with F(s)

8.1 Sampling Time

To determine an appropriate sampling time, the response of the plant in open loop is analysed. It is observed that the system's settling time is approximately 500 seconds. According to the empirical rule mentioned in [7], choosing 10% of the settling time a sampling time of 50 seconds is calculated. As a rule of thumb, it is possible to choose a sampling time lower than this last value.



Considering the bode diagram of the aggregate continuous transfer function F(s), the crossover pulse is $w_c = 0.0671$.

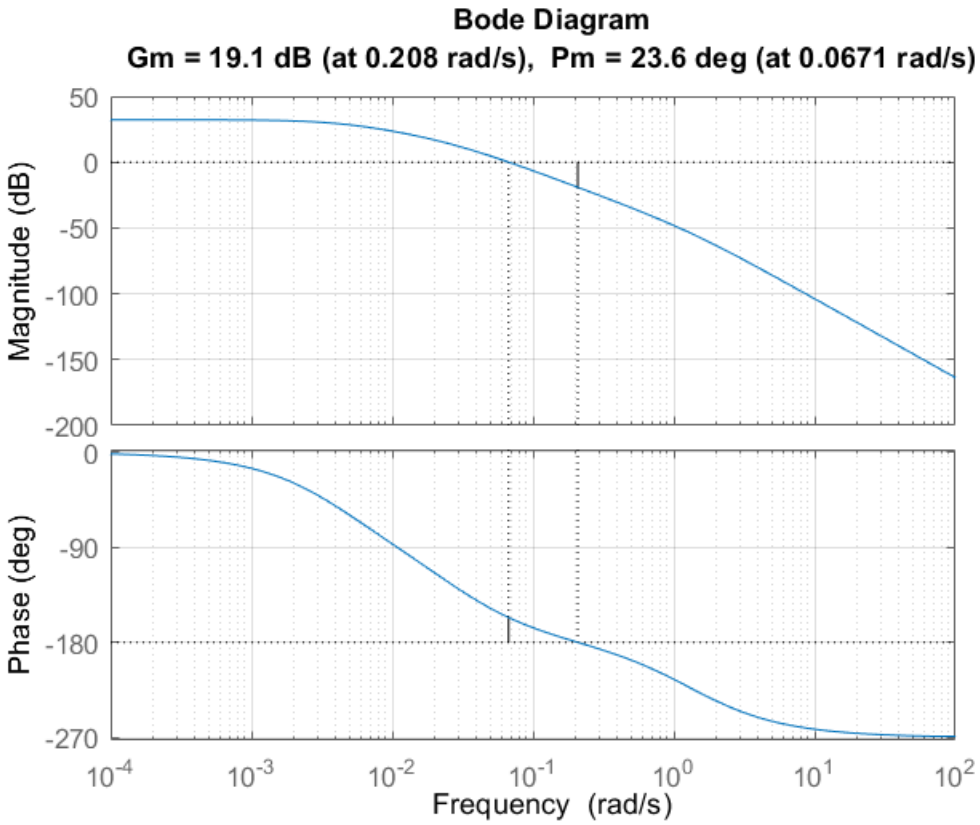


Figure 36 F(s) Bode Diagram

Due to the Shannon theorem, sampling pulse w_s needs to reflect the relation:

$$\omega_s > 2\omega_c$$

Since the relation between frequency and pulse $\omega = 2\pi f$, the sampling frequency needs to be:

$$f_s > \frac{\omega_c}{\pi} \rightarrow f_s > 0.0213$$

Considering the empirical indication in [7], the sampling frequency can be chosen within the range:

$$6\omega_c \leq f_s \leq 20\omega_c \rightarrow 0.4023 \leq f_s \leq 1.3411$$

Taking into account that typically a hydraulic system is slow [7], $f_s = 1\ sec$ is chosen.

This selection aligns with the rule of thumb mentioned earlier, as it is smaller than the calculated value of 50 seconds.

8.2 Pid Controller

A proportional–integral–derivative controller (PID controller or three-term controller) is a control loop mechanism employing feedback that is widely used in industrial control systems and a variety of other applications requiring Continuously modulated control. A PID controller continuously calculates an error value *e(t)* as the difference between a desired setpoint (SP) and a measured process variable (PV) and applies a correction based on proportional, integral, and derivative terms (denoted P, I, and D respectively), hence the name.

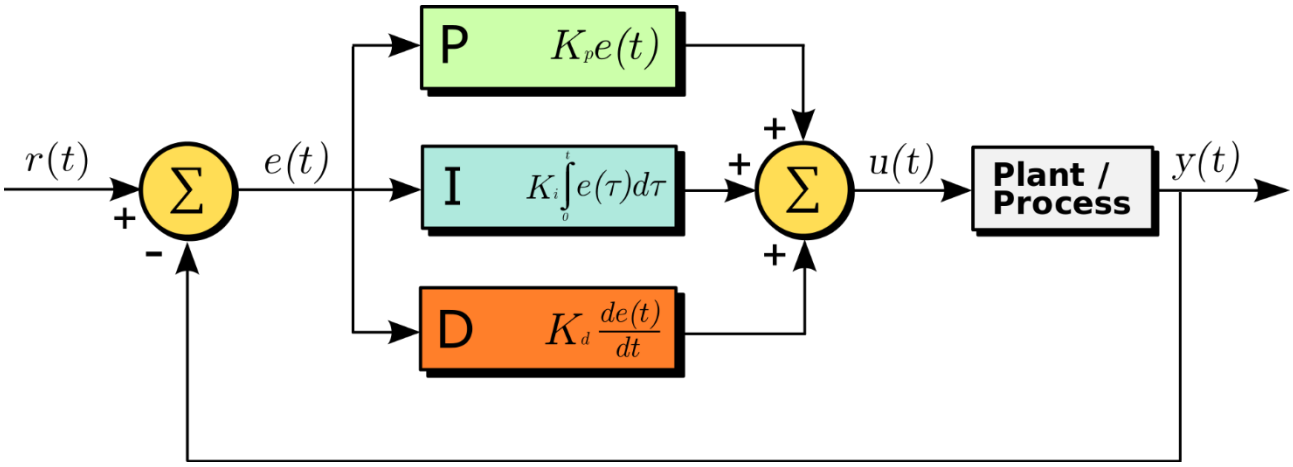


Figure 37 PID Scheme

The process of selecting the PID controller parameters to meet given performance specifications is performed by Matlab and Simulink on the discretized mathematical model. Thus, a discrete PID controller is evaluated to implement it on the STM32. The parameters found are represented in Table 1: PID parameters.

Parameter	Value
K_p	1.8410
K_i	0.0118
K_d	60.8833

Table 1: PID parameters

8.2.1 Pid Code

```
float PID_Control(PID_Param_t* PID, Tank_Param_t* tank){
    float epsilon = 1;
    PID -> prev_output = PID -> current_output;
    PID -> current_error = tank->level_current - tank->level_target;
    if(fabsf(PID -> current_error) > epsilon){
        PID -> integral += PID -> current_error*PID -> deltaT;
        PID -> derivative = (PID -> current_error - PID -> prev_error)/PID->deltaT;
        PID -> current_output = PID->Kp*PID->current_error + PID->Ki*PID ->integral + PID->Kd*PID->derivative;
        if(PID -> current_output > 100){
            PID -> saturation = true;
            PID -> current_output = 100;
        }
        else if(PID -> current_output < 0){
            PID -> saturation = true;
            PID -> current_output = 0;
        }
        else{
            PID -> saturation = false;
        }
    }
    PID -> prev_error = PID -> current_error;

    return PID ->current_output;
}
```

Figure 38 PID Function

This function will return the DC value to apply to the pump, starting from PID parameters and Tank 2 reference target and actual level, given by the sensor. An *epsilon* variable is defined in order to give

a tolerance to the error of 1 cm. Stored the previous output, the function computes the current error and if its more than our tolerance, we compute a new input; proportional, integral and derivative components are summed. A saturator limits the computed input in the 0-100 range and sets a flag in case of saturation, then the current error is stored in the previous error variable for the next iteration.

8.3 Lqi Controller

The Linear Quadratic Integral (LQI) control is a technique commonly utilized in control systems engineering. It extends the Linear Quadratic Regulator (LQR) by incorporating integral control to handle steady-state errors. By integrating the error signal over time, the LQI control aims to drive the system's output to the desired setpoint. The LQI design optimizes a cost function that incorporates both the quadratic performance criterion and an integral term. This allows for the reduction of steady-state errors and the improvement of tracking performance. Similar to the LQR, the LQI control employs state feedback and optimal control theory principles. It employs a state-space model of the system and determines the optimal state feedback gain matrix, to generate the control signal.

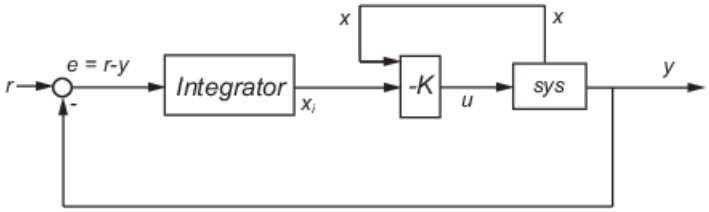


Figure 39 LQI Scheme

Similar to LQR, the tuning process of an LQI controller can also be performed in MATLAB to achieve the desired system performance. Compared to LQR, the matrix R and the matrix K in LQI consider the error with the reference signal in the last row for R matrix or last column for K matrix. Specified different reference signals, utilizing the state-space model in continuous time, by empirical changes to the weight matrix R can be made to tune the controller's performance. Like LQR, if the system is completely controllable and observable, choosing Q=C^T C uniquely determines the state feedback matrix K for LQI as well.

The different values of the matrix K=[k1 k2 k3] for various reference signals are represented in table 3.

Reference	k1	k2	k3
10	15.7345	23.1724	-0.3162
12.5	15.7345	23.1724	-0.3162
15	15.7345	23.1724	-0.3162

Table 2: LQI parameters

8.3.1 Lqi Code

```
float LQI_Control(LQI_Param_t* LQI, Tank_Param_t* tank1, Tank_Param_t* tank2 ){
    float epsilon = 1;
    LQI -> prev_output = LQI -> current_output;
    LQI -> current_error = tank2->level_current - tank2->level_target;
    if(fabsf(LQI -> current_error) > epsilon){
        LQI -> integral += LQI -> current_error*LQI -> deltaT;
        LQI ->current_output = - tank1->level_current*LQI->K1 - tank2->level_current*LQI->K2 + LQI->K3*LQI->integral;
        if(LQI -> current_output > 100){
            LQI -> saturation = true;
            LQI -> current_output = 100;
        }
        else if(LQI -> current_output < 0){
            LQI -> saturation = true;
            LQI -> current_output = 0;
        }
        else{
            LQI -> saturation = false;
        }

    }
    LQI -> prev_error = LQI -> current_error;
    return LQI ->current_output;

}
```

Figure 40 LQI Function

The function gives the DC value computed as output starting from LQI, Tank 1 and Tank 2 parameters. In this function, the error is computed similarly to the PID function; defined the *epsilon* tolerance, we compute the error's integral, then the DC value limited by the saturator.

9. Firmware Implementation

9.1 Ioc

The software implementation starts setting up all the timers, GPIOs and the I2C communication. First, two timers were set: the first one is a PWM generator which drives our pump through the L298N driver, the second one enables the actuation in a chosen time interval.

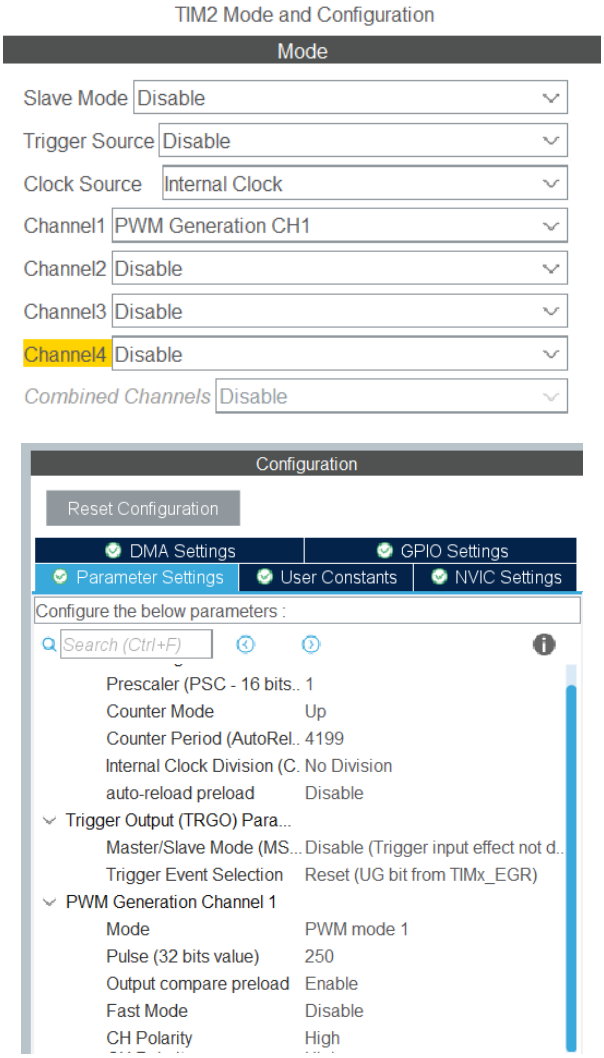


Figure 41 Timer 2 Configuration

Timer 2 has been chosen to generate a PWM signal on channel one starting from the internal clock as clock source. As discussed in the pump section, the Prescaler and the Counter Period are set respectively to 1 and 4199, in order to have a PWM frequency equal to 10 kHz. The Pulse value represents the Duty Cycle relative to the Counter Period value, and it gets modified by changing a value in the CCR register related to the PWM channel of our timer.

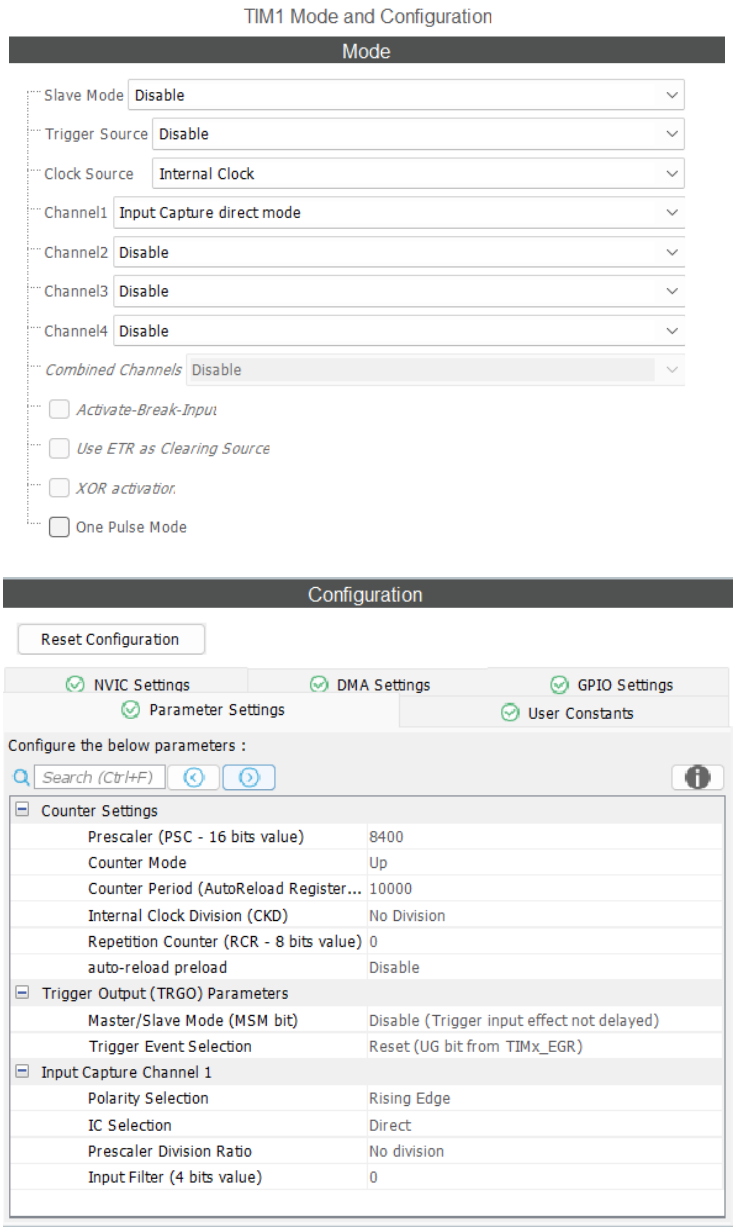


Figure 42 Timer 1 Configuration

Timer 1 has been configured as a clock for our actuation function: we need to compute an input every second, according to the chosen sample time for our plant. To achieve this, channel 1 has been set to Input Capture direct mode, referring to the internal clock source; Prescaler and Counter Period values have been respectively set to 8400 and 10000, to have a clock with a period of one second.

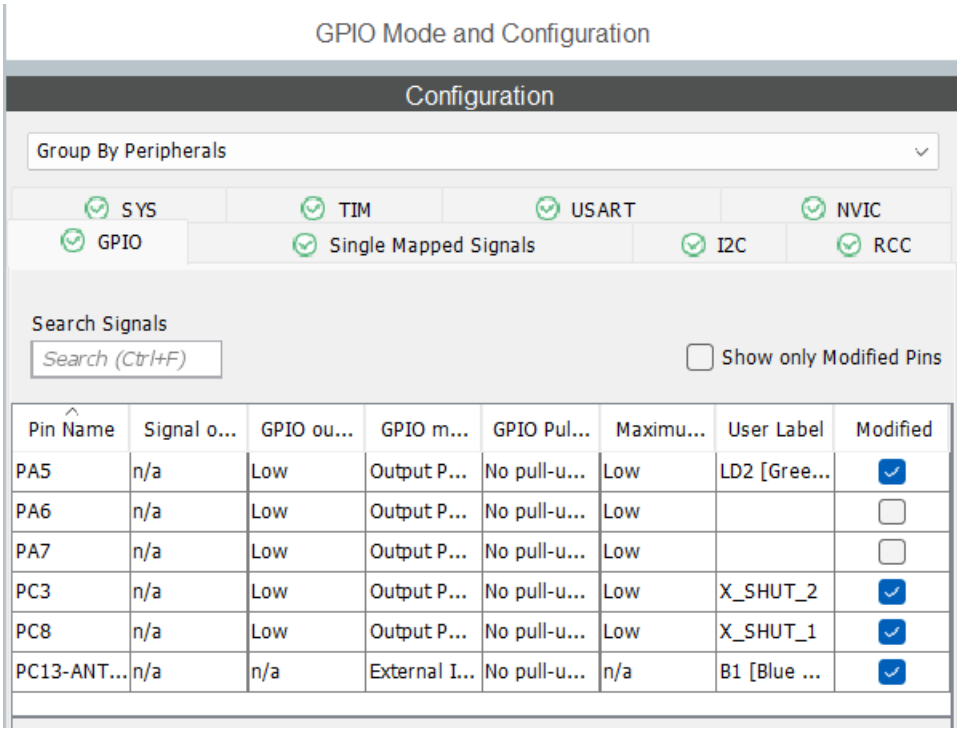


Figure 43 GPIOs Configuration

Four GPIOs outputs have been configured: two for the driver and two for the sensor. For the driver, PA6 and PA7 pins have been set as GPIO Outputs with default settings: those pins provide high- or low-level output to our driver, changing the motor's spinning direction or shutting down the pump (high-high or low-low level). For the sensor, PC3 and PC8 pins have been set as GPIO Outputs in order to control individually sensor's power consumption

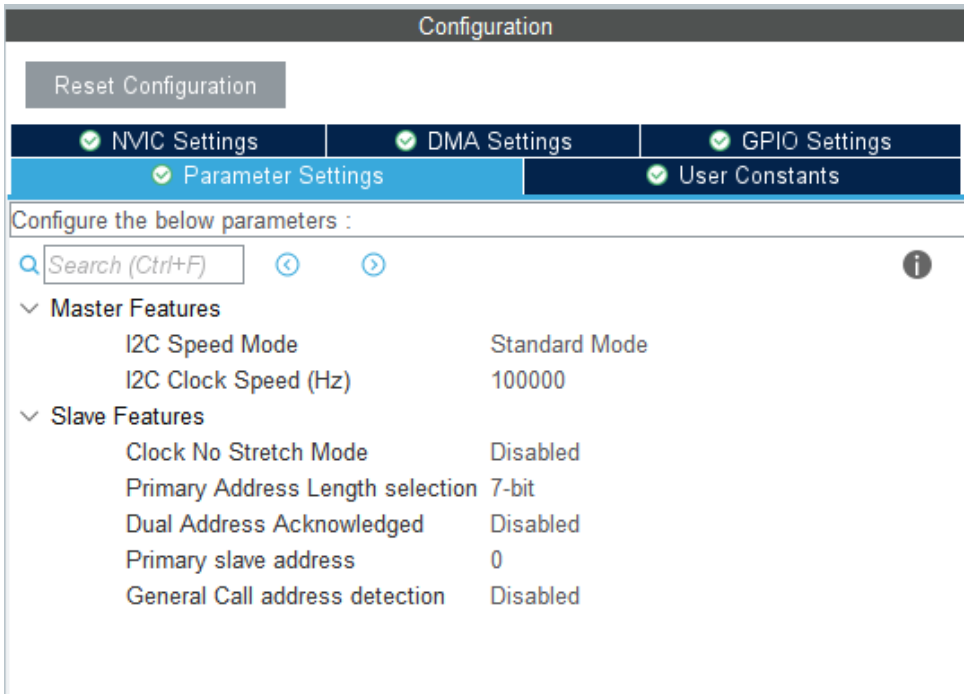


Figure 44 I2C Configuration

```
void PID_Init(PID_Param_t* PID){
    PID -> Kp = 1.8410;
    PID -> Ki = 0.0118;
    PID -> Kd = 60.8833;
    PID -> deltaT = 1;
    PID -> derivative = 0;
    PID -> integral = 0;
    PID -> current_error = 0;
    PID -> current_output = 0;
    PID -> prev_error = 0;
    PID -> prev_output = 0;
    PID -> saturation = false;

}
```

Figure 45 PID Init

All the structures are initialized to their default values in the *control.c* file: for instance, PID controller has *Kp*, *Ki* and *Kd* constants and *deltaT* initialized to the previously computed values, while all the other fields that can change their values during the firmware execution, are initialized to 0.

```
void Pump_Init(Pump_t* pump){
    pump -> PowerCurrent = 0;
}

void Tank1_Init(Tank_Param_t* tank){
    tank -> height = 25.5;
    tank -> level_current = 0;
    tank -> level_target = 0;
    tank -> sensor_height = 29.5;
}

void Tank2_Init(Tank_Param_t* tank){
    tank -> height = 25.5;
    tank -> level_current = 0;
    tank -> level_target = 10;
    tank -> sensor_height = 29.5;
}
```

Figure 46 Pump and Tanks Init

```
void LQI_Init(LQI_Param_t* lqi){
    lqi -> K1 = -15.7345;
    lqi -> K2 = -23.1724;
    lqi -> K3 = 0.3162;
    lqi -> current_error = 0;
    lqi -> current_output = 0;
    lqi -> deltaT = 1;
    lqi -> prev_error = 0;
    lqi -> prev_output = 0;
    lqi -> saturation = false;
}
```

Figure 47 LQI Init

9.2 Main Code

```
#include "main.h"
#include "i2c.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"

/* Private includes ----------
/* USER CODE BEGIN Includes */
#include <sensors.h>
#include "MovingAverageFilter.h"
#include <control.h>
/* USER CODE END Includes */

/* Private typedef -----------
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------
/* USER CODE BEGIN PD */
#define T_MAX 20000
#define T_S 1
```

Figure 48 Includes

```
/* USER CODE BEGIN PV */
float DC;
float* ptr_DC = &DC;

float t;
float* ptr_t = &t;

Pump_t Pump1;
Tank_Param_t Tank1;
Tank_Param_t Tank2;
PID_Param_t PID1;
LQI_Param_t LQI1;

volatile Pump_t* Pump1_ptr = &Pump1;
volatile Tank_Param_t* Tank1_ptr = &Tank1;
volatile Tank_Param_t* Tank2_ptr = &Tank2;
volatile PID_Param_t* PID1_ptr = &PID1;
volatile LQI_Param_t* LQI1_ptr = &LQI1;

/* Filter Handler */
MovingAverageFilter filter1;
MovingAverageFilter filter2;
/* USER CODE END PV */
```

Figure 49 Initial variables and parameters

The *main.c* file contains the main code that is executed by our microcontroller. First, all the header files are included in the main, in order to have access to all functions defined in the corresponding *.c* files.

In the main loop, we first define a struct for the sensors:

```
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* definition of sensor struct*/
    struct sensors array_sensor;
    array_sensor.Dev_array[0] = &array_sensor.vl53l0x_c[0];
    array_sensor.Dev_array[1] = &array_sensor.vl53l0x_c[1];

    uint32_t data1,data2;
    /*data from sensors*/
    struct data_sensors data_f_sensors;
```

Figure 50 Sensors struct

Then, all the peripherals get initialized (timers, USART, I2C, GPIOs), as well as our defined structures (PID, Pump, Sensors, Tanks, LQI)

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_USART2_UART_Init();
MX_I2C1_Init();
MX_TIM2_Init();
MX_TIM1_Init();
MX_TIM3_Init();
/* USER CODE BEGIN 2 */


Pump_Init(&Pump1);
Tank1_Init(&Tank1);
Tank2_Init(&Tank2);
PID_Init(&PID1);
LQI_Init(&LQI1);

/*Sensors Initialization*/
array_sensor.Dev_array[0]->I2cHandle = &hi2c1;
array_sensor.Dev_array[1]->I2cHandle = &hi2c1;
array_sensor.Dev_array[0]->I2cDevAddr =  array_sensor.Dev_array[1]->I2cDevAddr = original_addr;
printf("Sensor Init:\n\r");
Sensor_Init(array_sensor,HIGH_ACCURACY); // function from sensors.h
MovingAverageFilter_Init(&filter1, 2);
MovingAverageFilter_Init(&filter2, 2);
/* For PWM */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); /* PA6 filo nero*/
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET); /* PA7 filo arancione*/

// Start timer 1
HAL_TIM_Base_Start_IT(&htim1);
```

Figure 51 Peripherals and Struct Init

In the while loop, the reference value is set:

```
  while (1)
  {
    /* USER CODE END WHILE */
    /* USER CODE BEGIN 3 */
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);

    Tank2.level_target = 10;

    /*get data from sensors -- distance from sensors to liquid*/
    Get_Data_Sensors(array_sensor,&data1,&data2);

    /*get tanks liquid level -- difference from height of sensor and data sensors*/
    MapLevel(&data1,&data2,Tank1_ptr,Tank2_ptr,&level1,&level2);

    /*moving average filter for data get from sensors*/
    Tank1.level_current = MovingAverageFilter_Update(&filter1, level1);
    Tank2.level_current = MovingAverageFilter_Update(&filter2, level2);

  }
  /* USER CODE END 3 */
}
```

Figure 52 While loop

Data acquired from the sensor get stored in the *level_current* register of both tanks: those values are previously filtered by a moving average filter, in order to remove noisy fluctuations in the measured signal.

To compute and perform the actuation, a callback function in non-stopping mode on timer 1 has been implemented:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM1)
    {
        __disable_irq();
        HAL_GPIO_TogglePin(LD2_GPIO_Port,LD2_Pin);

        *ptr_DC = PID_Control(PID1_ptr, Tank2_ptr);
        //*ptr_DC = LQI_Control(LQI1_ptr,Tank1_ptr, Tank2_ptr);
        actuation_pump(Pump1_ptr,*ptr_DC);

        printf("%.2f,L1:%.3f,L2:%.3f,DC:%.3f\n",*(ptr_t),Tank1.level_current, Tank2.level_current,*(ptr_DC));

        *ptr_t = *ptr_t + T_S;
        __enable_irq();
    }

}
```

Figure 53 Timer callback

The LED on the board is toggled every Ts, the controller type is chosen changing the commented line of code: then, the DC value computed is passed to the *actuation_pump* function, which changes the timer2 CCR register. The pointer to t gets updated.

10. Results and Identification

Once the controller was tuned on mathematical model of the plant, we have seen the difference between the simulation result and real result obtained on plant.

Recall of implantation data:

- Ts = 1s
- Reference of water level for Tank 2 = 10 cm
- Max Liquid flow in input to Tank 1 = 39.1 cm^3/s

10.0.1 Pid Controller

K_p	K_i	K_d
1.8065	0.0118	60.8833

Simulation Response:

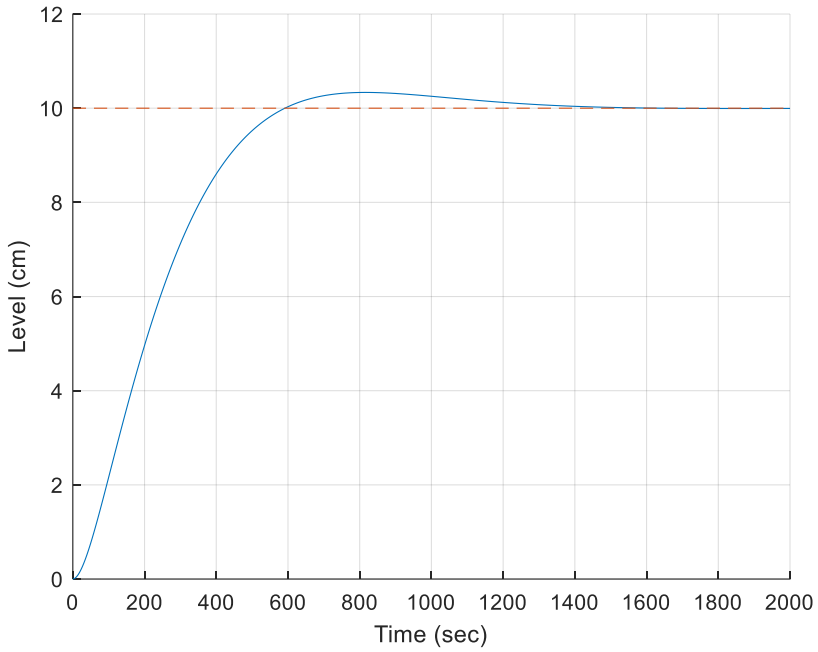


Figure 5421 Simulation Response PID

Plant Response

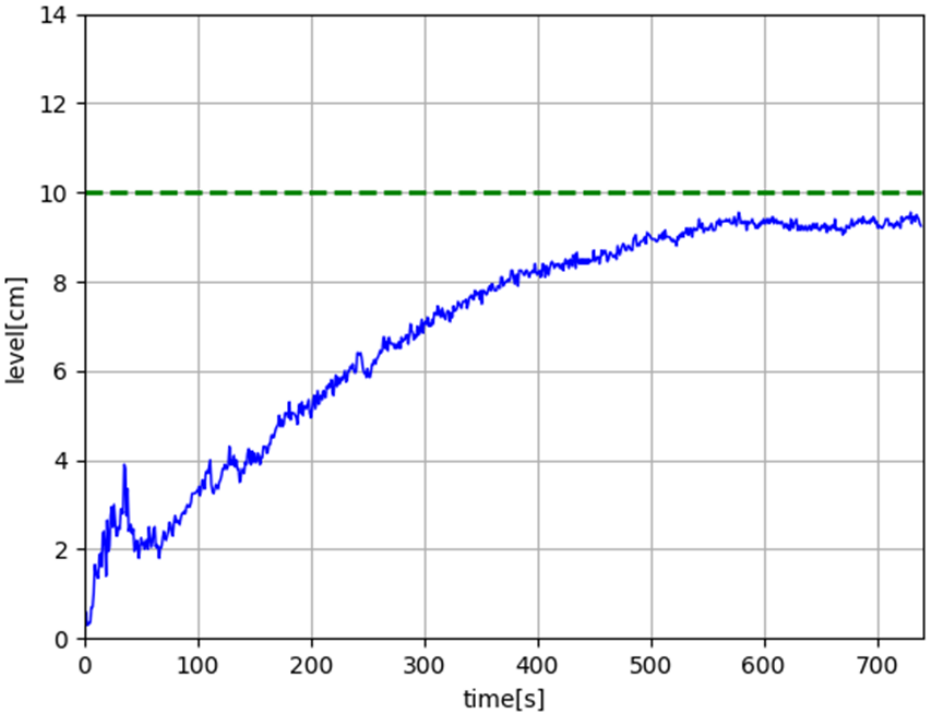


Figure 55 PID Tune on Mathematical Model

10.0.2 Lqi Controller

K1	K2	K3
15.7345	23.1724	-0.3162

Simulation Response

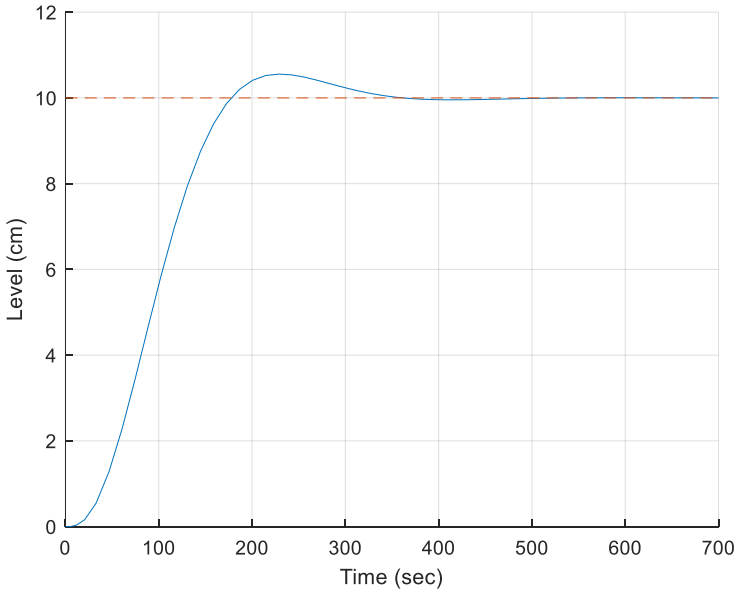


Figure 56 Simulation Response LQI

Real Response

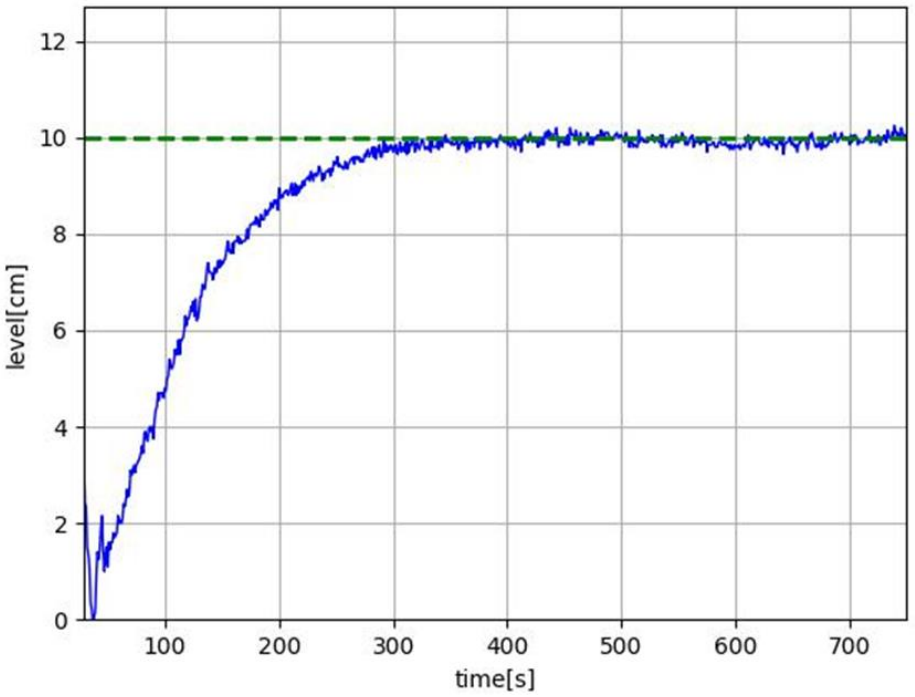


Figure 57 LQI Controller and Mathematical Model Response

10.1 System Identification

As can be seen, PID parameters based on model does not perform well as we expected in simulation, instead LQI parameters allow us to obtain a good response with a settling time of 350s. To tune a pid controller which provide us a step response close to the simulation it's necessary identify the system through System Identification Toolbox. Identification allows us to have a transfer function of the real system which is good approximation as compared to the mathematical model obtained by measurements of single components of the system.

Obviously, to perform the identification you need an input signal, in this case an APRBS was used to vary the DC in input to the driver. As output, we get the level of tank 2 and from figures below it possible to see system nonlinearity. The output data is not very precise as air bubbles formed on the liquid due to the fall from the upper hole, instead there are spikes in measurements that don't represent the actual water level.

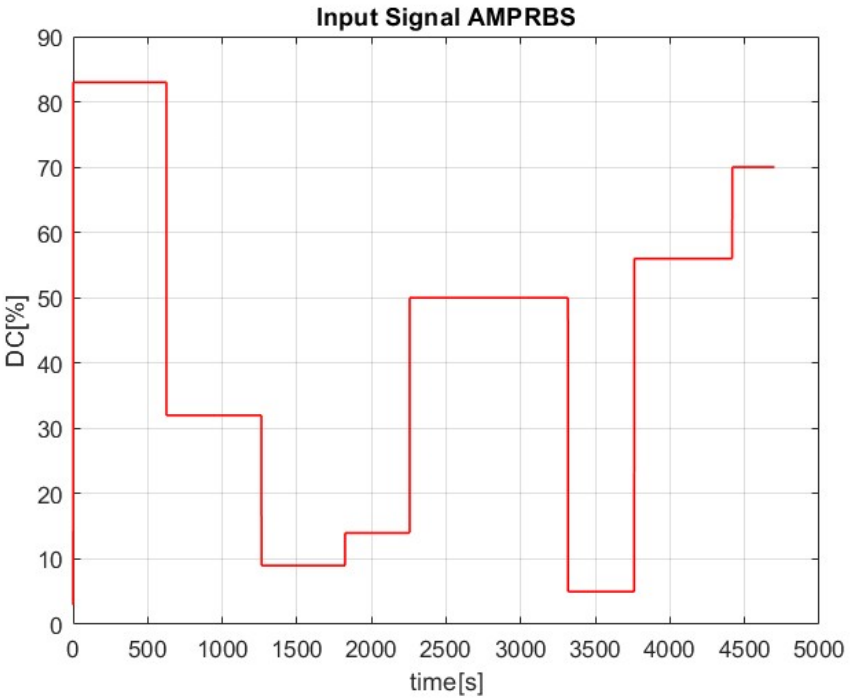


Figure 58 AMPRBS for System Identification – input signal

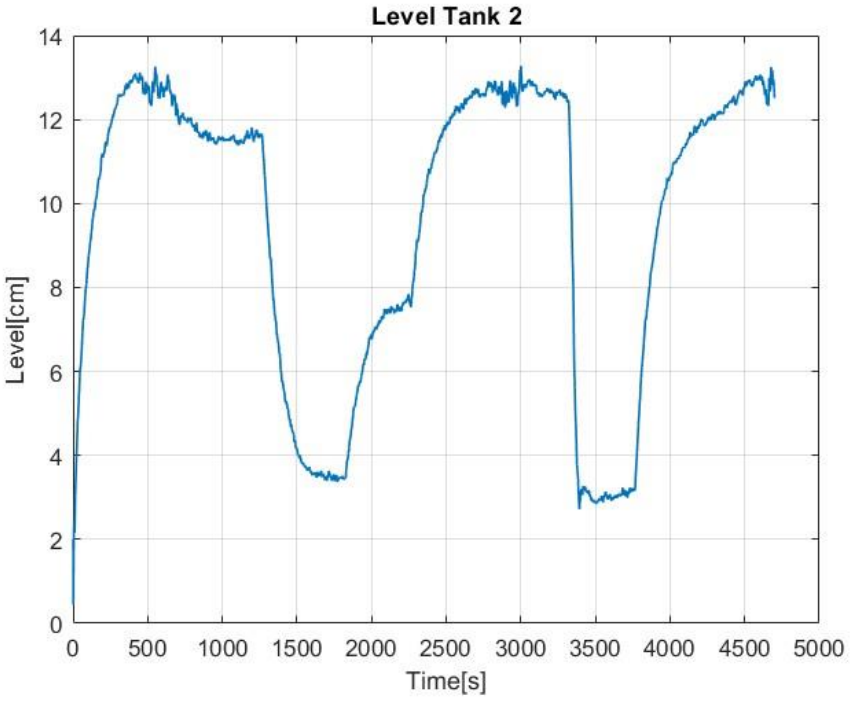


Figure 59 Tank 2 level – output signal

Transfer function from identification:

$$G(z) = \frac{0.006383}{(z^3 - 1.834z^2 + 1.729z - 0.8815)}$$

This result is obtained using Hammerstein - model for a SISO system.

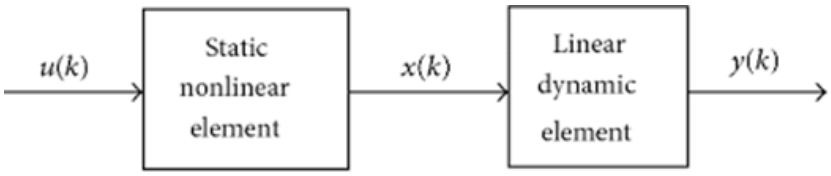


Figure 60 Hammerstein - Model

A PID controller was tuned using this transfer function and result was compared to real system response.

K_p	K_i	K_d
3	0.0757	1.48

Simulation Step Response

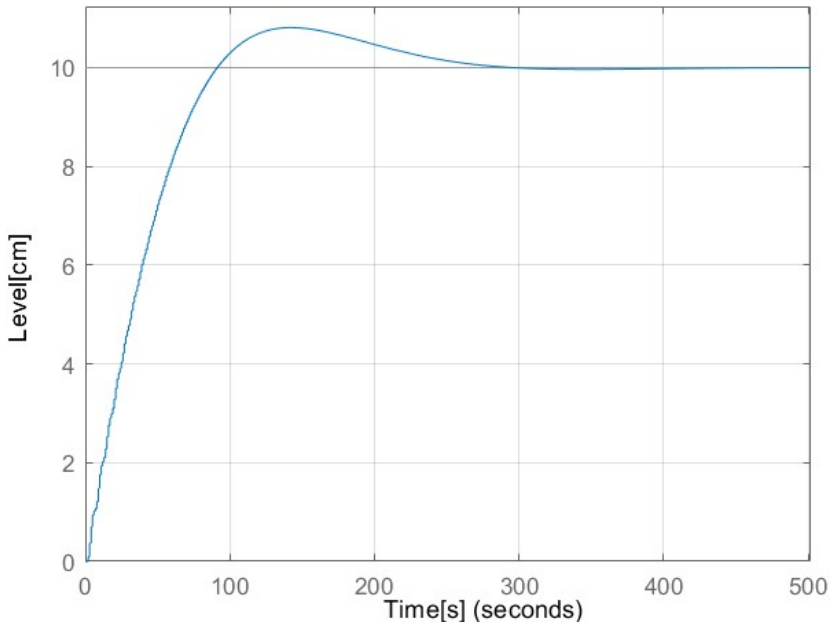


Figure 61 PID controller for identified system.

Real Step Response

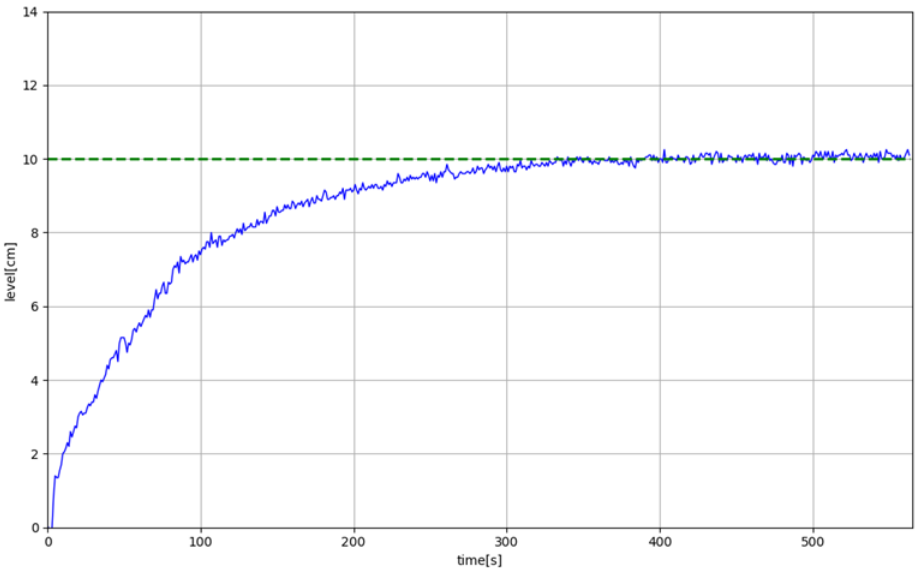


Figure 62 PID controller for identified system – Real response

As can be seen, real plant does not well as simulation result, but we have about the same settling time of 2%, that is 300s for simulation and 330s for real plant.

In conclusion, the two best performing controllers were validated. A disturbance has been introduced on the real plant, that is the exit hole of tank 2 has been blocked to increase the water level.

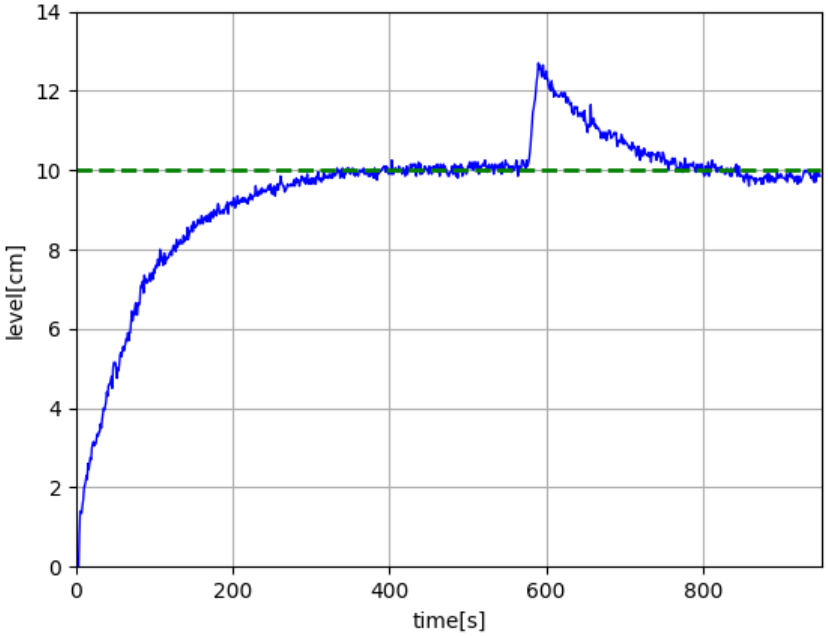


Figure 63 PID controller for identified system with disturbance.

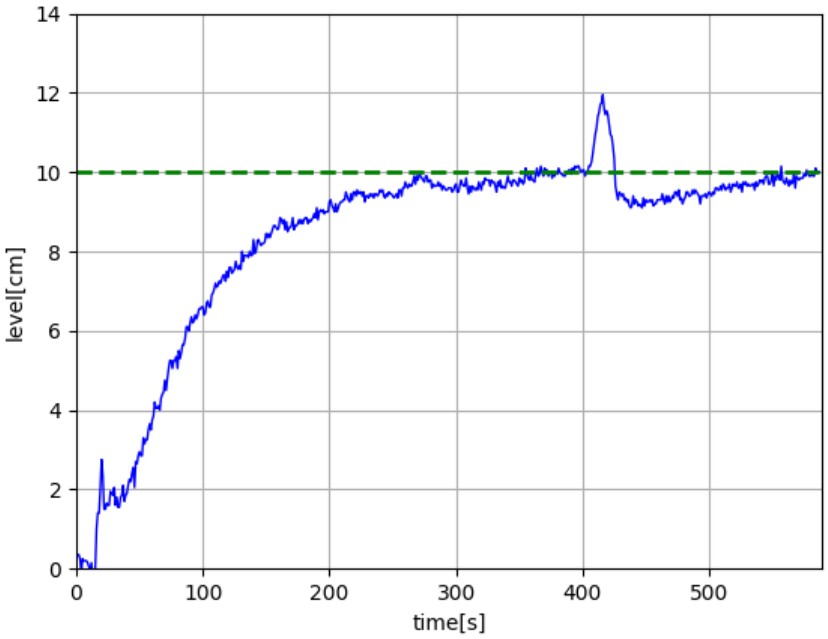


Figure 64 LQI controller for mathematical model with disturbance

As can we see from the images the PID controller, calibrated on the identified plant, can immediately return to reference with respect to the LQI controller.

11. Conclusions

In conclusion, the system was realized from both the physical and mathematical point of view. PID and LQI controllers were implemented. The tuning of these controllers was based on either the mathematical model of the system or the one obtained through identification. Subsequently, the performance of these controllers was evaluated. The number and type of sensors and actuators was sufficient for proper implementation of the control system and was fully compatible with the microcontroller and development environment. Based on the analysis and simulation results, it is evident that the PID and LQI parameters, derived from the mathematical model, did not meet the expected performance. In contrast, tuning based on the identified system demonstrated better performance, preserving the simulation results of the system.

Future developments may encompass the implementation of additional control strategies, including Model Predictive Control. Furthermore, extending the control objectives to include the regulation of liquid temperature and the level in Tank 1.

References

[1] VL53L0X Datasheet. https://www.st.com/resource/en/datasheet/vl53l0x.pdf

[2] VL53L0X API User Manual. https://www.st.com/resource/en/user_manual/um2039-world-smallest-timeofflight-ranging-and-gesture-detection-sensor-application-programming-interface-stmicroelectronics.pdf

[3] https://www.st.com/resource/en/application_note/an5843-water-and-liquid-level-measurement-using-vl53l5cx-timeofflight-8x8-multizone-sensor-with-wide-field-of-view-stmicroelectronics.pdf

[4] microcontroller - Is there an ideal PWM frequency for DC brush motors? - Electrical Engineering Stack Exchange

[5] Karl Johan Åström and Björn Wittenmark, *Computer-controlled Systems: Theory and Design*, 3rd ed. 1997.

[6] Giovanni Marro, *Controlli Automatici*, 5th ed. 2004.

[7] What causes hydraulics to run slow?: FPE Seals Ltd (no date) FPE Seals. Available at: https://www.fpeseals.com/faqs/what-causes-hydraulics-to-run-slow.

[8] https://it.mathworks.com/help/mpc/ug/choosing-sample-time-and-horizons.html